

Unit 12: Array Applications

CONTENTS

Objectives

Introduction

12.1 Searching

12.2 Sorting

12.3 Character Array

Summary

Keywords

Self Assessment

Answer for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- sorting techniques and its types.
- searching techniques and its types.
- Character Arrays

Introduction

One of the most important topics in DSA is sorting and searching. One of the most prevalent uses of computers nowadays is for storing and retrieving data. The amount of data and information kept and accessed by computer has grown over time, resulting in massive databases. To properly manage and process information in databases, a plethora of approaches and algorithms have been developed. Searching is the process of finding up a specific data record in a database. Sorting is the process of arranging records in a database. Combining sorting and searching is an important topic of research in computational algorithms. Both are crucial areas of research in data structures.

12.1 Searching

Searching is an operation or a strategy for locating a certain element or value inside a list. Whether or not the element being searched is found determines whether a search is successful or unsuccessful.

The practise of looking through the data in a data structure to see if a given value is present is known as searching. (As well as perhaps returning it.) For example, the contains method of the Array List searches the list for a given item and returns true or false.

Some of the standard searching technique that is being followed in data structure is listed below:

1. Linear Search
2. Binary Search

Linear search

Linear search is a simple and straightforward search strategy. In linear search, we look for an element or value in an array by traversing it from the beginning until the element or value we want is discovered.

It compares the element to be searched with all of the items in the array and returns the index of the element in the array if the element is successfully matched, otherwise it returns -1. When there are fewer elements in a list, Linear Search is used on unsorted or unordered lists.

Algorithm

Linear Search (Array A, Value x)

Step 1: Set i to 1

Step 2: if $i > n$ then go to step 7

Step 3: if $A[i] = x$ then go to step 6

Step 4: Set i to $i + 1$

Step 5: Go to Step 2

Step 6: Print Element x Found at index i and go to step 8

Step 7: Print element not found

Step 8: Exit

Pseudo code

```
procedure linear_search (list, value)
```

```
  for each item in the list
```

```
    if match item == value
```

```
      return the item's location
```

```
    end if
```

```
  end for
```

```
end procedure
```

Features of Linear Search Algorithm

1. It is used for unsorted and unordered small list of elements.
2. It has a time complexity of $O(n)$, which means the time is linearly dependent on the number of elements, which is not bad, but not that good too.
3. It has a very simple implementation

Complexity of algorithm

Complexity	Best Case	Average Case	Worst Case
Time	$O(1)$	$O(n)$	$O(n)$
Space			$O(1)$



Example: Let us take an example of an array $A[7] = \{5, 2, 1, 6, 3, 7, 8\}$. Array A has 7 items. Let us assume we are looking for 7 in the array. Targeted item=7.

Here, we have

A[7]={5,2,1,6,3,7,8}

X=7

At first, When i=0 (A[0]=5; X=7) not matched

i++ now, i=1 (A[1]=2; X=7) not matched

i++ now, i=2(A[2])=1; X=7)not matched

...

....

i++ when, i=5(A[5]=7; X=7) Match Found

Hence, Element X=7 found at index 5.

Linear search is rarely used practically. The time complexity of above algorithm is O(n).

int linearSearch(int values[], int target, int n)

```
{
    for(int i = 0; i < n; i++)
    {
        if (values[i] == target)
        {
            return i;
        }
    }
    return -1;
}
```



Lab Exercise: Linear search program

```
#include <stdio.h>

int main()
{
    int array[100], search, c, n;

    printf("Enter number of elements in array\n");
    scanf("%d", &n);

    printf("Enter %d integer(s)\n", n);

    for (c = 0; c < n; c++)
        scanf("%d", &array[c]);

    printf("Enter a number to search\n");
    scanf("%d", &search);
```

```

for (c = 0; c < n; c++)
{
    if (array[c] == search) /* If required element is found */
    {
        printf("%d is present at location %d.\n", search, c+1);
        break;
    }
}
if (c == n)
    printf("%d isn't present in the array.\n", search);

return 0;

```

Binary Search

Binary Search is used with sorted array or list. In binary search, we follow the following steps:

1. We start by comparing the element to be searched with the element in the middle of the list/array.
2. If we get a match, we return the index of the middle element.
3. If we do not get a match, we check whether the element to be searched is less or greater than in value than the middle element.
4. If the element/number to be searched is greater in value than the middle number, then we pick the elements on the right side of the middle element (as the list/array is sorted, hence on the right, we will have all the numbers greater than the middle number), and start again from the step 1.
5. If the element/number to be searched is lesser in value than the middle number, then we pick the elements on the left side of the middle element, and start again from the step 1.

Binary Search is useful when there are large number of elements in an array and they are sorted. So a necessary condition for Binary search to work is that the list/array should be sorted.

Features of Binary Search

1. It is great to search through large sorted arrays.
2. It has a time complexity of $O(\log n)$ which is a very good time complexity. It has a simple implementation.

Binary search is a fast search algorithm with run-time complexity of $\tilde{O}(\log n)$. This search algorithm is based on the divide-and-conquer strategy. The data collection should be in sorted form for this algorithm to perform effectively.

By comparing the collection's middle item, binary search looks for a specific piece.

If a match is found, the item's index is returned. The item is searched in the sub-array to the left of the middle item if the middle item is greater than the item. Otherwise, look for the item in the sub-array to the right of the centre item. This method is repeated on the subarray until the subarray's size is reduced to zero.

Algorithm

Step 1: Data list must be ordered list in ascending order.

Step 2: Probe middle of list

Step 3: If target equals list[mid], FOUND.

Step 4: If target < list[mid], discard 1/2 of list between list[mid] and list[last].

Step 5: If target > list[mid], discard 1/2 of list between list[first] and list[mid].

Step 6: Continue searching the shortened list until either the target is found, or there are no elements to probe.

Pseudo code

The pseudo code of binary search algorithms

A ← sorted array

n ← size of array

x ← value to be searched

Set lower Bound = 1

Set upper Bound = n

while x not found

if upper Bound < lower Bound

EXIT: x does not exists.

set mid Point = lower Bound + (upper Bound – lower Bound) / 2

if A[mid Point] < x

set lower Bound = mid Point + 1

if A[mid Point] > x

set upper Bound = mid Point - 1

if A[mid Point] = x

EXIT: x found at location mid Point

end while

end procedure



Example: C program to implement recursive Binary Search

```
#include <stdio.h>
```

```
// A recursive binary search function. It returns location of x in given array arr [l..r] is present, otherwise -1
```

```
int binary Search(int arr[], int l, int r, int x)
```

```
{
    if (r >= l) {
        int mid = l + (r - l) / 2;

        // If the element is present at the middle
        // itself
        if (arr[mid] == x)
            return mid;

        // If element is smaller than mid, then
        // it can only be present in left subarray
        if (arr[mid] > x)
            return binarySearch(arr, l, mid - 1, x);

        // Else the element can only be present
        // in right subarray
        return binarySearch(arr, mid + 1, r, x);
    }

    // We reach here when element is not
    // present in array
    return -1;
}

int main(void)
{
    int arr[] = { 2, 3, 4, 10, 40 };
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 10;
    int result = binarySearch(arr, 0, n - 1, x);
    (result == -1) ? printf("Element is not present in array")
                  : printf("Element is present at index %d",
                           result);
    return 0;
}
```

12.2 Sorting

A sorting algorithm is an algorithm that puts elements of a list in a certain order. The most used orders are numerical order and lexicographical order. Efficient sorting is important to optimizing the use of other algorithms that require sorted lists to work correctly and for producing human-readable input.

Sorting is the process of placing elements from a collection in some kind of order. For example, a list of words could be sorted alphabetically or by length. Efficient sorting is important to optimize the use of other algorithms that require sorted lists to work correctly.

Importance of sorting

To represent data in more readable format.

Optimize data searching to high level.

The most common sorting algorithms are:

Bubble Sort

Insertion Sort

Selection Sort

Quick Sort

Merge Sort

Heap sort

Shell Sort

25 30 15 12 16  12 15 16 25 30

Sorting algorithms are often classified by :

- * Computational complexity (worst, average and best case) in terms of the size of the list (N).

For typical sorting algorithms good behaviour is $O(N\log N)$ and worst case behavior is $O(N^2)$ and the average case behaviour is $O(N)$.

- * Memory Utilization

- * Stability - Maintaining relative order of records with equal keys.

- * No. of comparisons.

- * Methods applied like Insertion, exchange, selection, merging etc.

Sorting is a process of linear ordering of list of objects.

Sorting techniques are categorized into

Internal Sorting

External Sorting

Internal Sorting takes place in the main memory of a computer.

eg: - Bubble sort, Insertion sort, Shell sort, Quick sort, Heap sort, etc.

External Sorting, takes place in the secondary memory of a computer, Since the number of objects to be sorted is too large to fit in main memory.

eg: - Merge Sort, Multiway Merge, Polyphase merge.

Bubble Sort

The simplest sorting algorithm is bubble sort. It works by comparing each adjacent pair of elements and swapping them if they are out of order. It works by stepping through the list to be sorted multiple times, comparing two items at a time and exchanging them if they are out of order. The process is repeated until no swaps are required, indicating that the list is sorted. For large data sets, this approach is ineffective. This approach has an average and worst case time complexity of $O(n^2)$, where n is the number of elements..

**Example:**

6 4 2 >> 4 6 2

4 6 2 >> 4 2 6

4 2 6 >> 2 4 6

Algorithm

```

for i=N-1 to 2 {
  set swap flag to false
  for j=1 to i {
    if list[j-1] > list[j]
      swap list[j-1] and list[j]
      set swap flag to true
  }
  if swap flag is false, break. The list is sorted.
}

```

Selection Sort

Selection Sort finds the smallest element in the array and exchanges it with the element in the first position, it then finds the second smallest element and exchanges it with the element in the second position and continues this process until the entire list is sorted.

Algorithm

```

Step 1: Set MIN to location 0
Step 2: Search the minimum element in the list
Step 3: Swap with value at location MIN
Step 4: Increment MIN to point to next element
Step 5: Repeat until list is sorted

```

**Example:**

22 10 15 18 >> 10 22 15 18

10 22 15 18 >> 10 15 22 18

10 15 22 18 >> 10 15 18 22

10 15 18 22

**Example:**

arr[] = 64 25 12 22 11

// Find the minimum element in arr[0...4]

// and place it at beginning

11 25 12 22 64

// Find the minimum element in arr[1...4]

```
// and place it at beginning of arr[1...4]
11 12 25 22 64
```

```
// Find the minimum element in arr[2...4]
// and place it at beginning of arr[2...4]
11 12 22 25 64
```

```
// Find the minimum element in arr[3...4]
// and place it at beginning of arr[3...4]
11 12 22 25 64
```



Example: program for implementation of selection sort

```
#include <stdio.h>
```

```
void swap(int *xp, int *yp)
{
    int temp = *xp;
    *xp = *yp;
    *yp = temp;
}
```

```
void selectionSort(int arr[], int n)
{
    int i, j, min_idx;

    // One by one move boundary of unsorted subarray
    for (i = 0; i < n-1; i++)
    {
        // Find the minimum element in unsorted array
        min_idx = i;
        for (j = i+1; j < n; j++)
            if (arr[j] < arr[min_idx])
                min_idx = j;

        // Swap the found minimum element with the first element
        swap(&arr[min_idx], &arr[i]);
    }
}
```

```
/* Function to print an array */
void printArray(int arr[], int size)
{
    int i;
    for (i=0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

// Driver program to test above functions
int main()
{
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr)/sizeof(arr[0]);
    selectionSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);
    return 0;
}
```

Output:

Sorted array:
11 12 22 25 64

Merge Sort

Merge sort is a recursive algorithm that involves splitting and merging the array.

Algorithm

The algorithm works as follows:

1. Divide the array in half.
2. Recursively sort both halves.
3. Merge the halves back together.



Example: program for Merge Sort

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Merges two subarrays of arr[].
```

```
// First subarray is arr[l..m]
```

```
// Second subarray is arr[m+1..r]
```

```
void merge(int arr[], int l, int m, int r)
```

```
{
```

```
    int i, j, k;
```

```
int n1 = m - l + 1;
int n2 = r - m;

/* create temp arrays */
int L[n1], R[n2];

/* Copy data to temp arrays L[] and R[] */
for (i = 0; i < n1; i++)
    L[i] = arr[l + i];
for (j = 0; j < n2; j++)
    R[j] = arr[m + 1 + j];

/* Merge the temp arrays back into arr[l..r]*/
i = 0; // Initial index of first subarray
j = 0; // Initial index of second subarray
k = l; // Initial index of merged subarray
while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
    }
    else {
        arr[k] = R[j];
        j++;
    }
    k++;
}

/* Copy the remaining elements of L[], if there
are any */
while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

/* Copy the remaining elements of R[], if there
are any */
while (j < n2) {
    arr[k] = R[j];
```

```
        j++;
        k++;
    }
}

/* l is for left index and r is right index of the
sub-array of arr to be sorted */
void mergeSort(int arr[], int l, int r)
{
    if (l < r) {
        // Same as (l+r)/2, but avoids overflow for
        // large l and h
        int m = l + (r - l) / 2;

        // Sort first and second halves
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        merge(arr, l, m, r);
    }
}

/* UTILITY FUNCTIONS */
/* Function to print an array */
void printArray(int A[], int size)
{
    int i;
    for (i = 0; i < size; i++)
        printf("%d ", A[i]);
    printf("\n");
}

/* Driver code */
int main()
{
    int arr[] = { 12, 11, 13, 5, 6, 7 };
    int arr_size = sizeof(arr) / sizeof(arr[0]);

    printf("Given array is \n");
    printArray(arr, arr_size);
}
```

```

mergeSort(arr, 0, arr_size - 1);

printf("\nSorted array is \n");
printArray(arr, arr_size);
return 0;
}

```

Output

Given array is

12 11 13 5 6 7

Sorted array is

5 6 7 11 12 13

Quick Sort

Quick sort is an interesting algorithm because while its worst case is technically $O(N^2)$, in practice it is almost always $O(N \log N)$

Quick sort is a recursive algorithm based around the idea of choosing a pivot item and sorting around it.

Algorithm

1. "Randomly" choose an element from the array as your pivot.
2. Partition the array around your pivot, making sure that items less than the pivot are to the left of it and all items greater than or equal to the pivot are to the right of it.
3. Recursively sort both parts.

12.3 Character Array

Character array is collection of characters, stored under a single name. Character arrays can be initialized using string literals (Double quotes). One dimensional array of characters also known as a string.

Initializing Character Array

Char	abc[] =	"Program";
Data type	name of character array	String
char a[] = "Program";		

"Program" is a string.

String terminates with null character i.e. `'\0'`.

String "Program" has 8 characters.

```

char abc[] = {"hello"};
char abc[10] = {'h','e','l','l','o','\0'};
Char abc[] = {'h','e','l','l','o','\0'};
Char abc[10] = {"hello"};

```



Example: #include<stdio.h>

```
int main(){
    char a[]={'h','e','l','l','o','\0'};
    for(int i=0;i<=6;i++){
        printf("%c",a[i]);
    }
    return 0;
}
```

Output:

hello

Summary

Searching is the process of looking through the data contained in a data structure and determining if a specific value is present.

Binary Search Algorithm follows the Divide and Conquer strategy where it finds the item from the sorted list of items

In a linear search, array is traversed sequentially and each element is checked until a match is found

A Sorting process is used to rearrange a given array or elements based upon selected algorithm/sort function

Bubble sort, swap neighbours the larger items drop down while the smaller ones bubble up, in n-1 passes through the array

Keywords

- Driver code
- Algorithm
- Conquer strategy
- readable format
- high Level
- character array

Self Assessment

1. Every element in an array is searched against some searching key, special for
 - A. Linear search
 - B. Bubble sort
 - C. All of them
 - D. Binary search
2. A binary search algorithm is an algorithm that is used for
 - A. Divide and conquer method
 - B. Linear way to search values
 - C. Bubble sorting technique

-
- D. None of them
3. For finding value in an array which of the following technique is used?
- A. Binary search algorithm
 - B. Bubble sort
 - C. Linear search algorithm
 - D. All of them
4. Very slow way of sorting is.....
- A. Insertion sort
 - B. Heap sort
 - C. Bubble sort
 - D. Quick sort
5. Which of the following sorting algorithm is of divide and conquer type?
- A. Bubble sort
 - B. Insertion sort
 - C. Quick sort
 - D. Merge sort
6. If the given input array is sorted or nearly sorted, which of the following algorithm gives the best performance?
- A. Insertion sort
 - B. Selection sort
 - C. Quick sort
 - D. Merge sort
7. Binary search algorithm cannot be applied to ...
- A. sorted binary trees
 - B. sorted linear array
 - C. sorted linked list
 - D. pointer array
8. Which sorting method is external sorting?
- A. Bubble Sort
 - B. Merge Sort
 - C. Tree Sort
 - D. Insertion Sort
9. _____ sorting is useful in case of large amount of data.
- A. Merge
 - B. Bubble
 - C. Heap

D. Radix

10. A character constant is enclosed by?

- A. Left Single Quotes
- B. Right Single Quotes
- C. Double Quotes
- D. None of the above

11. A character array can be initialized using

- A. Floats value
- B. A string literal
- C. Integer values
- D. None of them

Answer for Self Assessment

- | | | | | |
|-------|------|------|------|-------|
| 1. A | 2. A | 3. A | 4. A | 5. C |
| 6. A | 7. C | 8. B | 9. D | 10. B |
| 11. B | | | | |

Review Questions

1. Write a program to implement a Linear Search Algorithm?
2. Differentiate between linear search and binary search.
3. Write down algorithm for binary search.
4. What is significance of sorting, give an example?
5. How selection sort is different from merge sort?
6. What is complexity in algorithm?



Further Readings

Books Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication, 2008

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi

Byron Gottfried, "Programming With C", Tata McGraw Hill Publishing Company Limited, New Delhi

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.



Web Links

www.en.wikipedia.org

www.web-source.net

www.webopedia.com

www.geeksforgeeks.org

www.tutorialspoint.com